Step 1 HR Driven Identity Lifecycle Lab Plan

This project will demonstrate how employee changes in an authoritative HR system can drive identity lifecycle management in Okta. I am building a custom HR application for the fictional JBC Cybersecurity organization, then connecting it to Okta to test Joiner, Mover, and Leaver workflows using synthetic employee records.

The goal is to model the separation of responsibilities found in enterprise environments: HR owns employment information, while Okta manages identity profiles, account lifecycle, and access through its supported policies and provisioning integrations.

Application and Infrastructure

The custom application, JBC HR, uses Django and Python on an Ubuntu VM within my Proxmox environment. Docker Compose runs the web application, a PostgreSQL database with persistent storage, and a background worker that applies approved changes at their effective times.

Employee records include an immutable employee ID, name, email, department, job title, manager, employment type, location, and employment dates. Separate submitter, approver, and auditor roles support controlled changes. A requester cannot approve their own request, and terminated employee records remain available for historical review.

This application has its own database and services, keeping it separate from my original CSV-based IGA lab.

Connecting HR to Okta

The integration uses an Okta Custom Identity Source to receive employment data and a separate OAuth API Services application to authenticate the connector.

Client authentication uses an RSA PEM private key with its corresponding public key registered in Okta. The connector also implements Demonstrating Proof of Possession, or DPoP, to bind access tokens to a separate proof key. Its connection check handles nonce challenges and signs fresh proofs for requests.

The next implementation phase will define the source attributes, configure mappings into Okta profiles, and correlate records using the immutable employee ID. A custom Python client will transport approved HR changes and track delivery outcomes. Initial testing will use one synthetic employee and a reviewed preview before live delivery.

Planned Lifecycle Tests

Lifecycle	Planned action	Verification
Joiner	Approve an eligible hire and deliver the employee record to Okta.	Confirm the correct account and profile, intended birthright access, and no duplicate account on a repeated run.
Mover	Approve a department, title, or manager change.	Confirm updated attributes, removal of obsolete access, and assignment of appropriate new access.
Leaver	Process an approved termination at its effective time.	Confirm Okta deactivation and downstream access removal wherever provisioning is configured, while retaining the HR record.

Mover testing will explicitly examine removal-before-grant ordering rather than assume native rules guarantee it. Leaver testing will verify actual account state rather than treat a “Terminated” profile attribute as proof of deactivation.

Current Checkpoint and Evidence Plan

The HR application is running, and synthetic hire requests have been submitted and approved. The live connection check has successfully verified OAuth authentication, DPoP handling, and read access to the Custom Identity Source.

Employee delivery remains disabled. End-to-end provisioning and lifecycle tests are still planned.

I will collect sanitized requests, approval records, import results, Okta account and group states, and audit events. Evidence will distinguish between a change applied in HR, an import processed by Okta, and access verified at the target. This will provide a traceable record of expected behavior, actual results, and any exceptions requiring recovery.

This is an independent home lab using synthetic identities, not a production HR system or a Saviynt tenant implementation.